Fundamentals of Data and Database Management System

Chapters 2 & 3: Database Architecture, Models, Relational Model and Normalization

Instructor: Tek Raj Pant

Presidential Graduate School, Westcliff University
DATA 210: Database Design & Analytics
BSIT Program

May 11, 2026

Lecture Outline

1. Chapter 2: Database Architecture & Models
2. Chapter 3: Relational Keys and Integrity Constraints
3. Normalization
4. Assignment 2 Deep Dive

Three-Level ANSI-SPARC Architecture

- The ANSI-SPARC architecture defines three levels of abstraction for a database:
  1. **External Level** (User Views)
  2. **Conceptual Level** (Community Logical View)
  3. **Internal Level** (Physical Storage)
- Separates user applications from physical database.
- Provides data independence and flexibility.

The Three Levels in Detail

External Level
- Describes how users see the data.
- Multiple views for different user groups.
- Hides rest of database.

Conceptual Level
- What data is stored and relationships among data.
- Entire database structure for the community.
- Independence from both physical and external layers.

Internal Level
- Physical representation on storage.
- File organizations, indexes, data compression.

Data Independence

- **Logical Data Independence**
  - Changes to conceptual schema do not affect external views.
  - Adding a new attribute to a table; existing views remain unchanged.
- **Physical Data Independence**
  - Changes to internal schema do not affect conceptual schema.
  - Moving data to a different disk or adding an index; applications unaffected.
- These layers make the relational model robust and maintainable.

Evolution of Data Models

- **Hierarchical** – Data organized as tree (parent/child).
- **Network** – Graph-based, many-to-many relationships via pointers.
- **Relational** – Tables with rows and columns, keys and constraints.
- **Object-Oriented** – Objects, classes, inheritance.
- **Object-Relational** – Extends relational with OO features.
- **NoSQL** – Document, key-value, columnar, graph; designed for scale.

Today's focus: The relational model – foundation of modern databases.

Relational Model Core Concepts

- **Relation** = a table with rows and columns.
- **Tuple** = a row (record) in a relation.
- **Attribute** = a column (field) of a relation.
- **Domain** = set of allowable values for an attribute.
- **Cardinality** = number of tuples.
- **Degree** = number of attributes.

Example: `CUSTOMER(CustID, Name, Phone, Member)`
Cardinality = 3, Degree = 4.

Keys in the Relational Model

- **Superkey**: Set of attributes that uniquely identifies a tuple.
- **Candidate Key**: Minimal superkey; cannot remove any attribute without losing uniqueness.
- **Primary Key**: Candidate key chosen by designer as main identifier.
- **Alternate Key**: Other candidate keys not chosen as primary.
- **Foreign Key**: Attribute(s) in one relation that reference the primary key of another relation; enforces referential integrity.

Keys Example – Diner Database

Relation	Attributes	Keys
CUSTOMER	CustID, Name, Phone, Member	Candidate/PK: CustID
ORDER	OrderID, CustID, OrderDate	Candidate/PK: OrderID; FK:
ORDERDETAIL	OrderID, ProductID, Qty, Price	Candidate/PK: {OrderID, Pro

- {OrderID, ProductID} is a composite primary key in ORDERDETAIL.
- ORDER.CustID references CUSTOMER(CustID).
- ORDERDETAIL.OrderID references ORDER(OrderID).

Integrity Rules

- **Entity Integrity**
  - Primary key must be unique and not null.
  - Guarantees each tuple is identifiable.
- **Referential Integrity**
  - Foreign key values must match an existing primary key value, or be null.
  - No “dangling references”.
  - In our diner: ORDER.CustID must exist in CUSTOMER.

Relational Database vs Flat File

Relational DBMS

- Data split into normalized tables.
- No or minimal duplication.
- Enforced integrity via keys.
- Complex queries using SQL joins.
- Better for multi-user, large datasets.

Flat File

- Single table with all fields.
- High data redundancy.
- No built-in constraints.
- Easy to create (Excel, CSV).
- Simple for small, single-purpose tasks.

Discussion: Which system do you feel more comfortable using and why?

Problems of Poorly Designed Tables

- **Insertion Anomaly**: Cannot insert certain data without other data (e.g., cannot add a product if no order exists).
- **Deletion Anomaly**: Deleting a tuple may remove required data unintentionally (losing customer info when last order deleted).
- **Update Anomaly**: Data inconsistency due to repeated information (e.g., customer phone stored in multiple rows).

Normalization is a process to organize data to reduce redundancy and avoid anomalies.

Functional Dependency (FD)

- If attribute B is **functionally dependent** on attribute A, then each value of A determines exactly one value of B.
- Notation: $A \rightarrow B$
- Examples:
  - $CustID \rightarrow Name, Phone$
  - $OrderID \rightarrow CustID, OrderDate$
  - $\{OrderID, ProductID\} \rightarrow Qty, Price$
- **Determinant**: The left-hand side attributes of an FD.
- FDs are the basis for normalization.

First Normal Form (1NF)

- A relation is in 1NF if:
  - All attributes contain only atomic (indivisible) values.
  - No repeating groups or arrays within a column.
- Example violation: A column `PhoneNumbers` storing multiple numbers as a string.
- Solution: Separate table or multiple rows.
- Most modern DBMS automatically enforce 1NF.

Second Normal Form (2NF)

- A relation is in 2NF if:
  - It is in 1NF, and
  - Every non-key attribute is fully functionally dependent on the *entire* primary key (no partial dependencies).
- Partial dependency: Non-key attribute depends on part of a composite primary key.
- Example: In ORDERDETAIL(OrderID, ProductID, Qty, Price, ProductName), ProductName depends on only ProductID $\rightarrow$ violates 2NF.
- Fix: Extract a PRODUCT relation with ProductID as key.

Third Normal Form (3NF)

- A relation is in 3NF if:
  - It is in 2NF, and
  - No transitive dependencies: a non-key attribute must not depend on another non-key attribute.
- Transitive dependency: $A \rightarrow B$ and $B \rightarrow C$ where B is not a key.
- Example: CUSTOMER(CustID, Name, ZipCode, City) with $ZipCode \rightarrow City$.
- Fix: Separate ZIPCODE table with ZipCode as key.

Boyce-Codd Normal Form (BCNF)

- Stronger version of 3NF.
- A relation is in BCNF if for every non-trivial functional dependency $X \rightarrow Y$, X is a superkey.
- BCNF catches anomalies when there are multiple overlapping candidate keys.
- In practice, most well-designed 3NF schemas also satisfy BCNF.
- Rarely violated in simple business databases.

When to Denormalize

- Normalization optimizes storage and consistency.
- Denormalization deliberately introduces redundancy to improve query performance.
- Typical use cases:
  - Data warehousing (star schemas).
  - Reporting tables that aggregate data from many joins.
  - Frequently accessed data where join overhead is high.
- Must manage redundancy with careful update procedures.

Recap: Week 1 Tables (Diner Case)

Table	Attributes
CUSTOMER	CustID, Name, Phone, Member
ORDER	OrderID, CustID, OrderDate
ORDERDETAIL	OrderID, ProductID, Quantity, UnitPrice

- Three normalized tables used to track customer orders.
- Relationships via primary and foreign keys.

Assignment 2 Part A: Identify All Keys

1. **Candidate Keys**
   - CUSTOMER: {CustID}, possibly {Phone} if unique.
   - ORDER: {OrderID}
   - ORDERDETAIL: {OrderID, ProductID}
2. **Primary Keys**
   - CUSTOMER: CustID
   - ORDER: OrderID
   - ORDERDETAIL: (OrderID, ProductID)
3. **Foreign Keys**
   - ORDER.CustID $\rightarrow$ CUSTOMER.CustID
   - ORDERDETAIL.OrderID $\rightarrow$ ORDER.OrderID

Assignment 2 Part B: Create a Single Tabulated Table

- The owner wants a **flat file** that shows all customer order details without joining.
- Normalized tables are joined to produce a single denormalized reporting table.
- New table combines attributes from CUSTOMER, ORDER, and ORDERDETAIL.
- Data may be duplicated: customer name and phone appear on every row.
- Resulting schema:
  `DinerReport(CustID, Name, Phone, Member, OrderID, OrderDate,`
  `ProductID, Quantity, UnitPrice)`
- Create this in MS Word or as a spreadsheet.

Part B Example Snapshot

CustID	Name	Phone	Member	OrderID	OrderDate	ProductID	Qty	UnitPrice
101	Alice	555-0101	Y	1001	2026-01-10	P01	2	5.99
101	Alice	555-0101	Y	1001	2026-01-10	P02	1	8.50
102	Bob	555-0202	N	1002	2026-01-11	P01	3	5.99

- Notice customer 101 appears twice – denormalized.
- Easy to read but contains redundancy.

Assignment 2 Part C: Add Rank with NULL for Non-Members

- The owner wants a **Rank** column that orders customer products by some metric (e.g., total spent per product per customer).
- **Rule**: Rank applies only to purchases made by loyalty program members.
- For non-member rows, the Rank value must be `NULL`.
- Approach:
  - Group by CustID, ProductID (or just per row context).
  - Compute rank based on Quantity or total (Quantity $\times$ UnitPrice).
  - Assign rank 1, 2, 3... for members; for non-members set Rank = NULL.
- This preserves the flat file structure and adds a new column.

Part C Example with Rank

CustID	Name	Phone	Member	OrderID	OrderDate	ProductID	Qty	UnitPrice	Rank
101	Alice	555-0101	Y	1001	2026-01-10	P01	2	5.99	1
101	Alice	555-0101	Y	1001	2026-01-10	P02	1	8.50	2
102	Bob	555-0202	N	1002	2026-01-11	P01	3	5.99	NULL
101	Alice	555-0101	Y	1003	2026-01-12	P01	4	5.99	1

- Rank can be computed per customer, per product, or overall – define clearly.
- Bob's row has Rank = NULL because he is not a member.

Key Takeaways – Relational Keys

- Every table must have a **primary key** to enforce entity integrity.
- **Foreign keys** implement relationships; they enforce referential integrity.
- Good key design is the first step toward normalization and data consistency.

Key Takeaways – Normalization

- Normalization minimizes update anomalies.
- 1NF: Atomic values.
- 2NF: No partial dependencies on composite keys.
- 3NF/BCNF: No transitive dependencies.
- Denormalization is a conscious trade-off for performance.

Relational vs Flat File in Practice

- Use **relational** databases for transactional systems that require consistency, concurrent access, and complex queries.
- Use **flat files** (denormalized tables) for quick analyses, small one-off reports, or when data sharing as CSV/Excel.
- Understanding both helps bridge the gap between database designers and business users.

Questions & Discussion

Discussion points:

- Experiences with relational vs flat file systems.
- How to decide on keys for a new database.
- Normalization challenges in real projects.
- Assignment clarifications for Part B and Part C.